

# KONSEP DAN RANCANGAN SISTEM

## MULTI-PRINCIPAL LEARNING MANAGEMENT SYSTEM (LMS)

### Dokumen Diskusi Pra-Implementasi

**Status:** Draft untuk pembahasan internal

**Tujuan:** Menyamakan konsep, kebutuhan bisnis, arsitektur, dan ruang lingkup sebelum implementasi menggunakan Antigravity.

---

## 1. LATAR BELAKANG

Perusahaan membutuhkan sebuah sistem Learning Management System (LMS) yang dapat digunakan untuk mengelola proses pembelajaran karyawan/tenaga kerja dari berbagai Principal dalam satu platform.

Saat ini data user dapat berasal dari berbagai sumber, salah satunya Odoo. Namun LMS tidak boleh bergantung sepenuhnya kepada Odoo.

Oleh karena itu, LMS dirancang sebagai **aplikasi standalone** yang memiliki:

- Database sendiri
- User interface sendiri
- Business logic sendiri
- Course management sendiri
- Learning progress sendiri
- Assessment sendiri
- Certificate sendiri
- Notification sendiri
- Reporting sendiri
- Import/export sendiri
- Audit trail sendiri

Odoo hanya menjadi **salah satu sumber data eksternal** melalui API.

Dengan pendekatan ini, apabila Odoo sedang mengalami gangguan, LMS tetap dapat digunakan.

---

## 2. TUJUAN SISTEM

Tujuan utama LMS:

1. Mengelola user dari berbagai Principal.
  2. Menggunakan NIK/KTP Number sebagai identitas utama seseorang.
  3. Memungkinkan satu orang memiliki beberapa riwayat pekerjaan/NIP.
  4. Mengelola Principal, Position, Department, dan Employment History.
  5. Mengelola Course, Module, Lesson, Material, Assessment, dan Certificate.
  6. Melakukan assignment course secara otomatis berdasarkan Principal dan Position.
  7. Mencatat seluruh progress pembelajaran.
  8. Menyediakan pre-test dan post-test.
  9. Menghasilkan sertifikat secara otomatis.
  10. Mengirimkan notifikasi melalui Email dan WhatsApp.
  11. Menyediakan import Create dan Import Update secara terpisah.
  12. Menyediakan export data.
  13. Menyediakan reporting dan dashboard.
  14. Terintegrasi dengan Odoo melalui API.
  15. Memiliki audit trail yang lengkap.
  16. Tetap cepat dan stabil walaupun jumlah user, course, progress, dan transaksi meningkat secara signifikan.
- 

## 3. PRINSIP UTAMA SISTEM

Ada beberapa prinsip yang harus dianggap sebagai **non-negotiable requirement**.

### 3.1 One Person = One LMS User

Satu orang hanya boleh memiliki satu User di LMS.

Identitas utama:

#### **NIK / KTP Number**

NIK bersifat:

- UNIQUE
- Mandatory
- Primary identity
- Tidak boleh menghasilkan duplicate user

Email juga harus UNIQUE.

---

## 4. KONSEP IDENTITAS USER

### 4.1 Mengapa NIP bukan Primary Key?

NIP tidak digunakan sebagai identitas utama karena seorang user dapat berpindah Principal atau mengalami perubahan NIP.

Contoh:

**User A**

NIK:

327xxxxxxxxxxxxxx

Riwayat:

| NIP    | Principal | Position    | Status   |
|--------|-----------|-------------|----------|
| AMK001 | Xiaomi    | Team Leader | Inactive |
| AKP099 | Samsung   | Team Leader | Active   |

Walaupun memiliki dua NIP, orang tersebut tetap:

**1 User LMS**

Bukan 2 user.

---

## 5. EMPLOYMENT HISTORY

Untuk menyelesaikan masalah tersebut, informasi pekerjaan dipisahkan dari User.

Entity utama:

**User**

dan

**Employment History**

Employment History minimal memiliki:

- User
- NIP
- Principal
- Position
- Department
- Start Date
- End Date
- Status
- Source
- Created At
- Updated At

Dalam kondisi normal:

**1 user hanya memiliki 1 Employment History ACTIVE.**

Riwayat lama tidak dihapus.

---

## 6. PRINCIPAL HISTORY

Selain Employment History, sistem memiliki:

### Principal History

Tujuannya untuk mengetahui perjalanan user antar Principal.

Contoh:

```
User
|
|— Principal History
|   |— Xiaomi
|   |   |— Start Date
|   |   |— End Date
|   |— Samsung
|   |   |— Start Date
|   |   |— Active
|— Employment History
```

└─ NIP AMK001  
└─ NIP AKP099

History tidak boleh dihapus karena dibutuhkan untuk:

- Audit
- Reporting
- Learning History
- Certificate History
- Analisa perpindahan Principal

## 7. PERPINDAHAN PRINCIPAL

Contoh:

User saat ini:

NIK : 123456  
NIP : AMK001  
Principal : Xiaomi  
Position : Team Leader

Kemudian pindah ke:

NIP : AKP099  
Principal : Samsung  
Position : Team Leader

Sistem harus:

1. Mencari user berdasarkan NIK.
2. Tidak membuat user baru.
3. Menutup Employment History lama.
4. Menutup Principal History Xiaomi.
5. Membuat Employment History baru.
6. Membuat Principal History Samsung.
7. Menjadikan Samsung sebagai Principal aktif.
8. Mengubah current NIP menjadi AKP099.
9. Mengubah current Principal menjadi Samsung.
10. Mengubah Position/Department jika berubah.
11. Menjalankan Automatic Course Assignment.
12. Mempertahankan seluruh history lama.

Semua proses tersebut harus dilakukan dalam **satu database transaction**.

Jika salah satu proses gagal, perubahan harus dibatalkan untuk mencegah data setengah berubah.

---

## 8. SUMBER DATA USER

LMS harus mendukung beberapa sumber data.

### 8.1 Odoo API

Odoo menjadi salah satu sumber data.

Data yang dapat diterima:

- NIK
- NIP
- Name
- Email
- Phone
- Principal
- Position
- Department
- Status

### 8.2 Manual Entry

Admin dapat membuat atau mengubah user secara manual.

### 8.3 Import Create

Untuk membuat data baru.

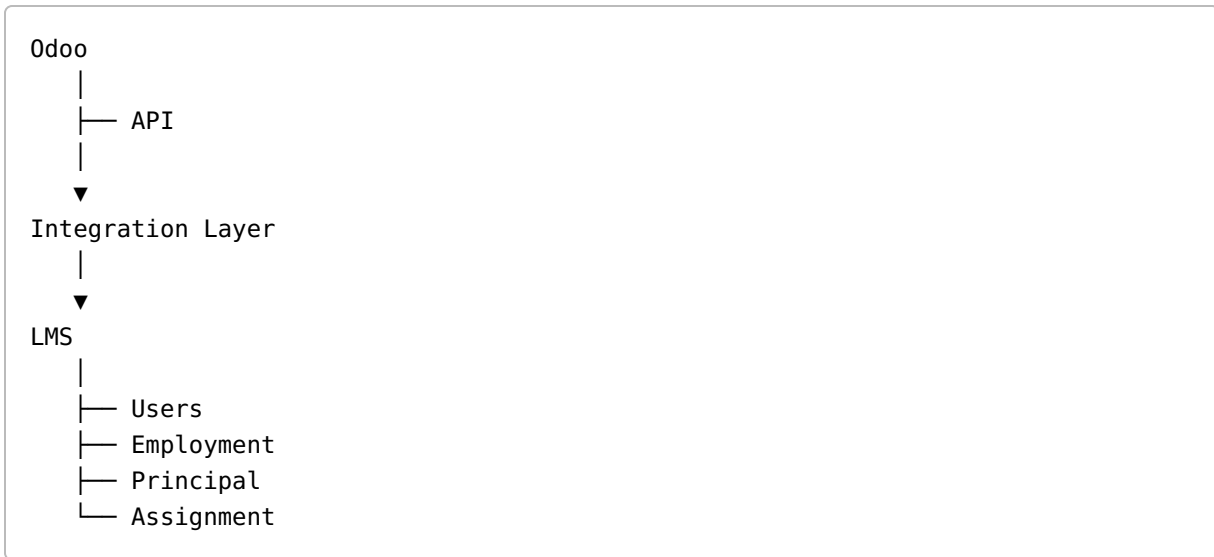
### 8.4 Import Update

Untuk memperbarui data existing.

### 8.5 Future API

Arsitektur harus memungkinkan sumber lain ditambahkan di masa depan.

Contoh:



## 9. ATURAN IMPORT

Import Create dan Import Update harus **dua proses yang berbeda**.

Tidak boleh dibuat satu menu:

Create or Update

### 9.1 Import Create

Tujuan:

#### CREATE ONLY

Jika unique key sudah ada:

#### ERROR

Tidak boleh melakukan update.

### 9.2 Import Update

Tujuan:

#### UPDATE ONLY

Jika unique key tidak ditemukan:

## ERROR

Tidak boleh membuat data baru.

---

## 10. ALUR IMPORT

Setiap import mengikuti alur:

```
graph TD; A[Upload File] --> B[Template Validation]; B --> C[Required Field Validation]; C --> D[Format Validation]; D --> E[Duplicate Check]; E --> F[Unique Key Check]; F --> G[Reference Check]; G --> H[Business Rule Validation]; H --> I[Preview]; I --> J[Admin Confirmation]; J --> K[Processing]; K --> L[Result]; L --> M[Audit Log];
```

Sebelum proses dijalankan, admin harus dapat melihat:

- Total Row
- Valid
- Error
- Warning

Untuk Update, idealnya ditampilkan:

| Field     | Old Value | New Value |
|-----------|-----------|-----------|
| Principal | Xiaomi    | Samsung   |
| NIP       | AMK001    | AKP099    |

---

## 11. IMPORT ERROR REPORT

Jika terdapat error, sistem harus menghasilkan informasi:

- Row Number
- Unique Key
- Status
- Error Message

Contoh:

```
Row 27
NIK: 123456
Status: ERROR
Error: NIK already exists
```

Error rows dapat diexport kembali.

---

## 12. IMPORT BATCH LOG

Setiap proses import memiliki ID.

Contoh:

```
IMP-2026-000001
```

Informasi:

- Import ID
- Menu
- Import Type
- File Name
- Uploaded By
- Upload Date
- Total

- Success
  - Failed
  - Status
- 

## 13. MASTER DATA

Master data utama:

### Organization

- Principal
- Position
- Department

### User

- User
- Employment History
- Principal History

### Learning

- Course
- Module
- Lesson
- Lesson Material
- Assignment Rule

### Assessment

- Question Bank
- Question
- Assessment
- Assessment Attempt
- Assessment Answer

### Certificate

- Certificate Template
- Certificate Field
- Certificate

## Notification

- Notification Template
  - Notification Setting
  - Notification Log
- 

## 14. KONSEP PRINCIPAL

Principal merupakan entity bisnis.

Contoh:

```
Xiaomi  
Samsung  
Oppo  
Vivo
```

Satu Principal dapat memiliki:

- Banyak User
  - Banyak Course
  - Banyak Position
  - Banyak Assignment Rule
- 

## 15. HUBUNGAN PRINCIPAL DAN COURSE

Course tidak hanya dimiliki oleh satu Principal.

Relationship:

**Principal ↔ Course = Many-to-Many**

Contoh:

```
Xiaomi  
├─ Course 1  
├─ Course 2  
└─ Course 3  
  
Samsung
```

- └─ Course 2
- └─ Course 4
- └─ Course 5

## 16. PENTING: PRINCIPAL TIDAK OTOMATIS MEMBERIKAN SEMUA COURSE

Hanya karena user berada di Principal Xiaomi, bukan berarti semua Course Xiaomi otomatis diberikan.

Assignment dapat berdasarkan:

- Principal
- Course
- Position
- Department jika diperlukan
- Effective Date
- Expiry Date
- Status

## 17. ASSIGNMENT RULE

Contoh Xiaomi:

| Course   | Position    |
|----------|-------------|
| Course 1 | Team Leader |
| Course 2 | Team Leader |
| Course 3 | Supervisor  |
| Course 4 | Sales       |

Maka:

Team Leader

- Course 1
- Course 2

Supervisor

- Course 3

Sales  
→ Course 4

## 18. ALL POSITION

Assignment Rule harus mendukung:

### ALL POSITION

Contoh:

Principal : Xiaomi  
Course : Product Knowledge  
Position : ALL POSITION

Artinya semua user aktif Xiaomi eligible mendapatkan Course tersebut.

## 19. AUTOMATIC ASSIGNMENT ENGINE

Sistem memiliki:

### Automatic Course Assignment Engine

Engine mengevaluasi:

- Current Principal
- Current Position
- Department jika relevan
- Active Assignment Rules
- Effective Date
- Expiry Date

Trigger:

1. User baru dibuat.
2. User pindah Principal.
3. User berubah Position.
4. Assignment Rule diaktifkan.
5. Course dihubungkan dengan Principal.
6. User menjadi eligible setelah import.

7. Odoo synchronization selesai.

Sistem harus mencegah duplicate enrollment.

---

## 20. COURSE STRUCTURE

Struktur:

```
Course
|
├── Module 1
|   ├── Lesson 1
|   ├── Lesson 2
|   └── Lesson 3
|
├── Module 2
|   ├── Lesson 4
|   └── Lesson 5
|
├── Post-Test
|
└── Certificate
```

Course dapat memiliki:

- Description
- Thumbnail
- Duration
- Passing Grade
- Certificate Rule
- Status

Passing grade dapat memiliki:

**Default Global**

dan

**Override per Course**

---

## 21. MATERIAL PEMBELAJARAN

Lesson mendukung:

1. Uploaded Video
2. YouTube Video
3. External Video URL
4. PowerPoint
5. PDF

Arsitektur harus extensible untuk material baru di masa depan.

---

## 22. VIDEO

Uploaded video harus diproses dengan pendekatan yang mendukung:

- Streaming
- Resume
- Progress Tracking
- Completion Tracking

Arsitektur yang direkomendasikan:

**FFmpeg + HLS + Object Storage + CDN**

Video tidak sebaiknya dilayani langsung melalui aplikasi Laravel.

---

## 23. KONTROL VIDEO

Untuk video tertentu dapat diberlakukan aturan:

- Progress harus tercatat.
- User dapat melanjutkan dari posisi terakhir.
- Completion hanya ketika kriteria terpenuhi.
- Playback speed dapat dikunci apabila requirement tersebut diberlakukan.

Tujuan:

User tidak dapat sekadar membuka video lalu langsung dianggap selesai.

---

## 24. SEQUENTIAL LEARNING

Lesson dapat dibuat sequential.

Contoh:

```
Lesson 1
↓
100% Completed
↓
Lesson 2 Unlock
↓
100% Completed
↓
Lesson 3 Unlock
```

Lesson berikutnya terkunci sampai lesson sebelumnya memenuhi completion criteria.

---

## 25. STATUS LESSON

Lesson:

- NOT STARTED
  - IN PROGRESS
  - COMPLETED
- 

## 26. STATUS COURSE

Course:

- NOT STARTED
  - IN PROGRESS
  - COMPLETED
  - FAILED
  - EXPIRED
  - CANCELLED
  - PRINCIPAL INACTIVE
-

## 27. LEARNING PROGRESS

Progress harus menyimpan:

- User
- Course
- Module
- Lesson
- Percentage
- Last Watched Position
- Started At
- Last Access
- Completed At
- Status

Progress harus otomatis tersimpan.

Jika browser tertutup atau koneksi terputus:

```
User kembali
  ↓
Continue Learning
  ↓
Resume dari posisi terakhir
```

Tetapi completion rules tetap harus diberlakukan.

---

## 28. LEARNING HISTORY

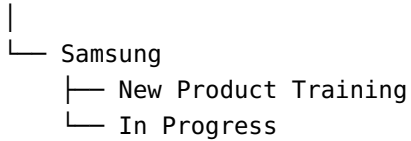
Learning History harus menjadi data historis.

Jika user pindah Principal:

**history lama tidak dihapus.**

Contoh:

```
User
|
├─ Xiaomi
|   ├─ Product Knowledge
|   └─ Completed Certificate
```



Dashboard utama menampilkan kondisi current Principal.

Learning History tetap menyimpan semua riwayat.

---

## 29. ASSESSMENT

Course dapat memiliki:

- Pre-Test
- Post-Test

Keduanya optional.

---

## 30. QUESTION BANK

Pertanyaan dapat berasal dari:

1. Reusable Question Bank
2. Manual Course Questions

Minimal mendukung:

- Multiple Choice
- True / False
- Multiple Answer

Future:

- Essay
  - Matching
  - Ordering
  - Image Question
- 

## 31. PASSING GRADE

Contoh:

Global default:

**80**

Course tertentu dapat override:

```
Course A = 80
Course B = 75
Course C = 90
```

---

## 32. CERTIFICATE ELIGIBILITY

Certificate dapat mensyaratkan:

1. Course 100%
2. Post-Test selesai jika enabled
3. Score  $\geq$  Passing Grade
4. Certificate Rule aktif

Contoh:

```
Course Completed
+
Post-Test Passed
+
Score  $\geq$  80
↓
Certificate Eligible
```

---

## 33. CERTIFICATE

Certificate menggunakan:

### **Certificate Template**

Template memiliki dynamic fields.

Contoh:

- User Name

- NIK
  - NIP
  - Principal
  - Position
  - Department
  - Course Name
  - Score
  - Completion Date
  - Certificate Number
  - Issue Date
  - Expiry Date
  - Custom Field
- 

## 34. CERTIFICATE DESIGNER

Admin dapat membuat template sertifikat.

Output:

- PDF
- PNG/JPG

Certificate Number harus UNIQUE.

---

## 35. HISTORICAL CERTIFICATE SNAPSHOT

Ini sangat penting.

Ketika sertifikat diterbitkan, data harus disimpan sebagai snapshot.

Contoh:

Saat sertifikat dibuat:

```
NIP      : AMK001
Principal : Xiaomi
Position  : Team Leader
```

Kemudian user pindah ke Samsung.

Sertifikat lama tetap:

NIP : AMK001  
Principal : Xiaomi  
Position : Team Leader

Tidak boleh berubah menjadi data terbaru.

---

## 36. PUBLIC CERTIFICATE VERIFICATION

Setiap certificate memiliki:

- Unique certificate number
- Public verification URL
- QR Code

User dapat:

**Download Certificate**

dan

**Copy Verification URL**

Orang lain dapat melakukan verifikasi tanpa login.

Informasi yang ditampilkan harus dibatasi hanya pada informasi yang memang diperlukan.

---

## 37. NOTIFICATION

Channel:

- Email
- WhatsApp

Event:

- Course Assigned
  - Course Reminder
  - Deadline Reminder
  - Course Completed
  - Assessment Result
  - Certificate Issued
-

## 38. NOTIFICATION HIERARCHY

Setting notification:

```
Global
↓
Principal
↓
User
```

Setting yang lebih spesifik dapat override setting yang lebih umum.

---

## 39. NOTIFICATION LOG

Setiap notification dicatat:

- Notification ID
- User
- Type
- Channel
- Status
- Created At
- Sent At
- Error

Status:

- PENDING
- SENT
- FAILED
- CANCELLED

Notification diproses menggunakan queue.

---

## 40. WHATSAPP

Integrasi WhatsApp harus menggunakan:

**Official WhatsApp Business API atau approved provider.**

Tidak menggunakan WhatsApp Web automation sebagai fondasi sistem.

---

## 41. USER ROLES

Minimal:

### **SUPER ADMIN**

Full access.

### **LMS ADMIN**

Mengelola:

- User
- Principal
- Position
- Department
- Course
- Assignment
- Assessment
- Certificate
- Notification
- Reports
- Integration

### **PRINCIPAL ADMIN**

Hanya dapat mengakses Principal yang diberikan.

### **LEARNER**

Hanya mengakses:

- Course
  - Lesson
  - Test
  - Progress
  - History
  - Certificate
  - Profile
-

## 42. LEARNER DASHBOARD

Dashboard learner menampilkan:

- Current Principal
  - Current Position
  - Assigned Courses
  - Continue Learning
  - Progress
  - Pending Test
  - Recent Completion
  - Certificates
- 

## 43. ADMIN DASHBOARD

Admin dashboard dapat menampilkan:

- Total Users
- Active Users
- Total Principal
- Total Course
- Course Progress
- Completion Rate
- Failed Assessment
- Certificates Issued
- Pending Learning
- Notification Status
- Import Status

Dashboard harus tetap ringan walaupun data besar.

---

## 44. REPORTING

Minimum report:

### 1. User Progress

- User
- NIK
- Current NIP
- Principal
- Position

- Course
- Progress
- Status
- Score
- Completion Date

## 2. Course Progress

- Course
- Principal
- Total Learners
- Not Started
- In Progress
- Completed
- Failed
- Average Score

## 3. Principal Report

Filter:

- Principal
- Position
- Course
- Date
- Status

## 4. Learning History

## 5. Assessment Result

## 6. Certificate Report

---

# 45. REPORTING PERFORMANCE

Report tidak boleh selalu menjalankan query berat langsung ke tabel transaksi.

Untuk data besar dapat digunakan:

- Aggregate table
- Materialized View
- Background processing
- Scheduled aggregation
- Cache

Tujuannya agar dashboard/report tidak memperlambat sistem transaksi.

---

## 46. AUDIT LOG

Setiap perubahan penting harus dicatat.

Minimal:

- Date/Time
- User
- Action
- Menu
- Record
- Field
- Old Value
- New Value
- Source
- Performed By
- IP jika diperlukan
- Request/API Reference jika diperlukan

Contoh:

```
2026-09-04 10:30
Admin: admin01
Action: UPDATE
Menu: User
NIK: 123456
Field: Principal
Old: Xiaomi
New: Samsung
Source: IMPORT UPDATE
```

---

## 47. SOURCE TRACKING

Data harus mengetahui asalnya.

Source:

- ODOO
- MANUAL

- IMPORT
- API

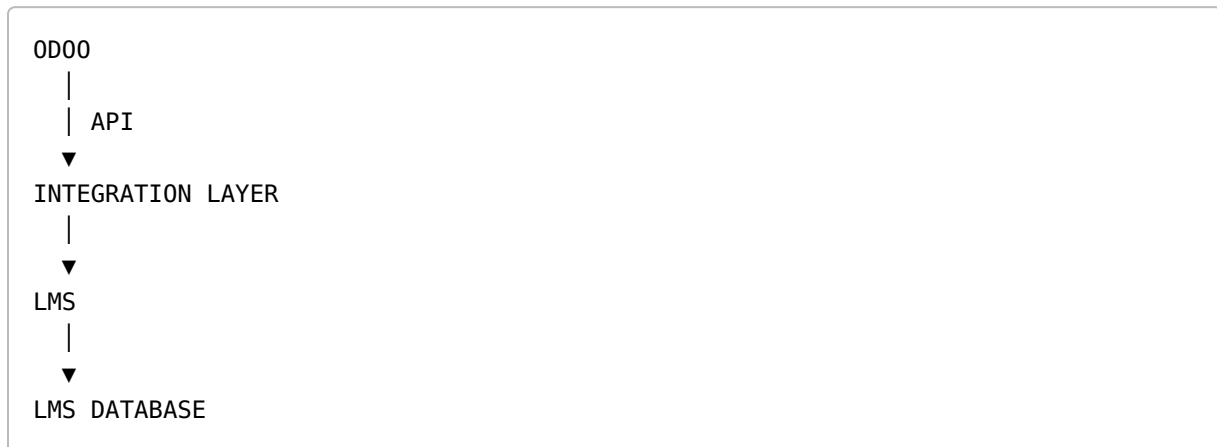
Juga dapat disimpan:

- Created Source
- Updated Source
- Last Sync

## 48. ODOO INTEGRATION

Odoo bukan database utama LMS.

Arsitektur:



LMS harus tetap berjalan ketika Odoo tidak tersedia.

## 49. ODOO USER MATCHING

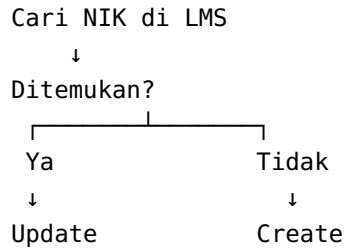
Matching user dilakukan berdasarkan:

**NIK**

Bukan NIP.

Logika:





Jika NIP berubah:

**update Employment History**

bukan membuat User baru.

---

## 50. ODOO SYNCHRONIZATION

Sync sebaiknya asynchronous.

Contoh:

```
graph TD; A[Odoo] --> B[API Request]; B --> C[Integration Queue]; C --> D[Process]; D --> E[Normalize Data]; E --> F[Update LMS]; F --> G[Assignment Engine]; G --> H[Notification];
```

Flowchart illustrating the Odoo synchronization process:

- Odoo
- ↓
- API Request
- ↓
- Integration Queue
- ↓
- Process
- ↓
- Normalize Data
- ↓
- Update LMS
- ↓
- Assignment Engine
- ↓
- Notification

Jangan membuat learner request bergantung langsung pada Odoo.

---

## 51. FIELD OWNERSHIP

Tidak semua field harus selalu ditimpa Odoo.

Harus ada aturan:

### Field Ownership / Sync Policy

Contoh:

| Field           | Odoo   | LMS    |
|-----------------|--------|--------|
| NIK             | Master | Read   |
| NIP             | Master | Read   |
| Name            | Master | Read   |
| Principal       | Master | Read   |
| Course Progress | -      | Master |
| Assessment      | -      | Master |
| Certificate     | -      | Master |

Hal ini harus disepakati sebelum integrasi dibuat.

---

## 52. EXPORT

Data harus dapat diexport minimal ke:

- XLSX
- CSV

Export berlaku untuk master data dan transaction/history sesuai kebutuhan.

---

## 53. LARGE DATA PROCESSING

Import/export data besar tidak boleh diproses dalam satu HTTP request.

Gunakan:

```
Request
↓
Create Job
↓
Queue
```

↓  
Worker  
↓  
Process in Chunks  
↓  
Result

Admin dapat melihat status:

QUEUED  
PROCESSING  
COMPLETED  
FAILED

## 54. ARSITEKTUR TEKNIS YANG DIREKOMENDASIKAN

Untuk tahap awal disarankan menggunakan:

### Backend

Laravel 13 + PHP 8.4+

### Frontend

Vue 3 + TypeScript + Vite + Tailwind CSS

### Database

PostgreSQL

### Cache / Queue

Redis

### Queue Monitoring

Laravel Horizon

## Web Server

Nginx

## Object Storage

S3-compatible Object Storage

## Video Processing

FFmpeg

## Video Streaming

HLS

## CDN / Security

Cloudflare

## Containerization

Docker

---

# 55. ARSITEKTUR: MODULAR MONOLITH

Pada tahap awal tidak perlu langsung menggunakan microservices.

Rekomendasi:

### Modular Monolith

Contoh:

```
LMS Application
|
├── Identity Module
├── User Module
├── Principal Module
├── Organization Module
├── Course Module
└── Assignment Module
```

```
|— Learning Module
|— Assessment Module
|— Certificate Module
|— Notification Module
|— Import Module
|— Reporting Module
|— Integration Module
|— Audit Module
```

Keuntungannya:

- Lebih sederhana
- Lebih cepat dikembangkan
- Lebih mudah maintenance
- Lebih mudah debugging
- Tetap dapat scale secara horizontal
- Modul dapat dipisahkan menjadi service jika suatu saat benar-benar diperlukan

---

## 56. DATABASE

PostgreSQL menjadi:

### Source of Truth

Minimal entity:

```
users
principals
positions
departments
employment_histories
principal_histories
courses
course_principals
modules
lessons
lesson_materials
assignment_rules
assignment_rule_positions
enrollments
lesson_progress
learning_histories
question_banks
questions
```

```
course_questions
assessments
assessment_attempts
assessment_answers
certificate_templates
certificate_fields
certificates
notification_templates
notification_settings
notification_logs
import_batches
import_rows
audit_logs
api_sync_logs
```

---

## 57. DATABASE RULES

Database harus memiliki constraint, bukan hanya validasi aplikasi.

Contoh:

```
users.nik UNIQUE
users.email UNIQUE
principal.code UNIQUE
position.code UNIQUE
department.code UNIQUE
course.code UNIQUE
module.code UNIQUE
lesson.code UNIQUE
certificate.number UNIQUE
```

History penting tidak boleh di-hard delete.

Gunakan:

- Status
- Archive
- Soft Delete jika sesuai

## 58. PERFORMANCE PRINCIPLES

Sistem harus dirancang dengan asumsi data akan terus bertambah.

Prinsip:

- Hindari SELECT \*
- Hindari N+1 query
- Gunakan pagination
- Gunakan cursor/keyset pagination untuk data besar
- Index berdasarkan query nyata
- Gunakan EXPLAIN ANALYZE
- Jangan melakukan pekerjaan berat di HTTP request
- Gunakan queue
- Gunakan Redis
- Gunakan object storage untuk file besar
- Gunakan CDN untuk video
- Gunakan aggregate untuk reporting
- Jangan menyimpan progress video setiap detik ke database

---

## 59. TARGET PERFORMANCE

Target awal yang dapat digunakan sebagai benchmark:

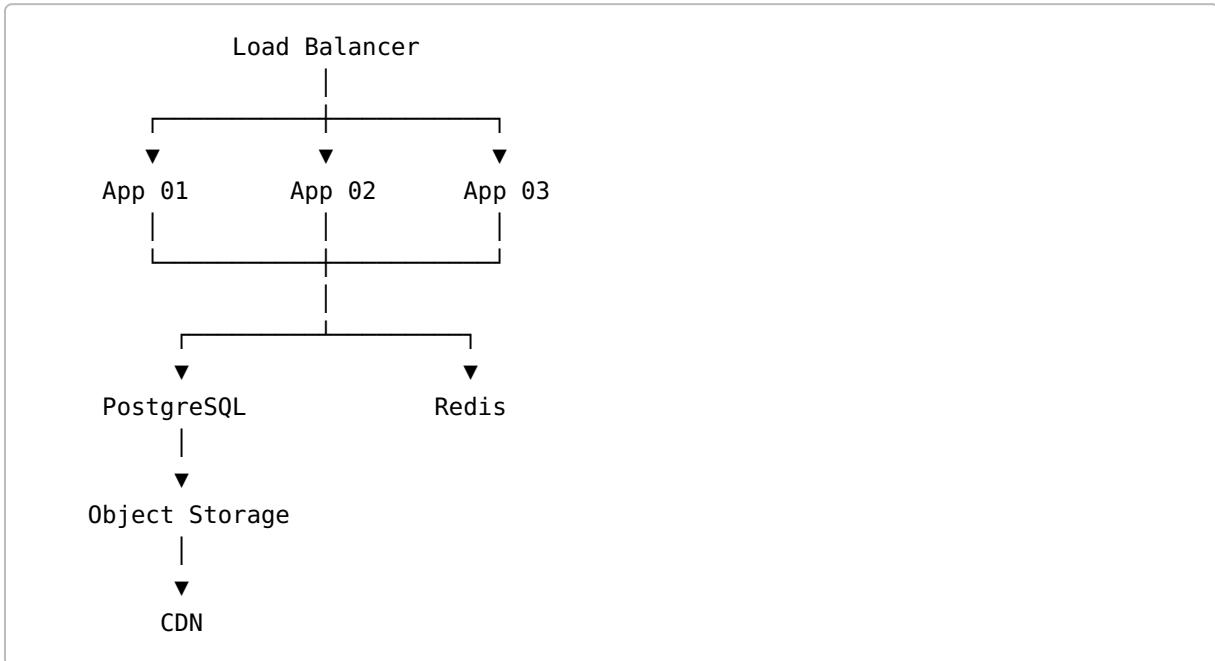
| Area               | Target    |
|--------------------|-----------|
| Normal API P50     | < 100 ms  |
| Normal API P95     | < 300 ms  |
| Normal API P99     | < 800 ms  |
| Dashboard P95      | < 1 detik |
| Course Listing P95 | < 500 ms  |
| Critical DB Query  | < 300 ms  |
| Error Rate         | < 0.1%    |
| Availability       | >= 99.9%  |

Angka tersebut adalah **target engineering**, bukan jaminan performa.

---

## 60. SCALABILITY

Application server harus stateless sehingga nantinya dapat dibuat:



Jika user meningkat, application server dapat ditambah tanpa mengubah konsep aplikasi.

## 61. FILE STORAGE

File besar seperti:

- Video
- PDF
- PowerPoint
- Certificate

tidak disimpan sebagai database binary utama.

Gunakan:

## Object Storage

Database hanya menyimpan metadata/reference.

## 62. SECURITY FILE

File tidak boleh terbuka melalui path yang tidak terkontrol.

Gunakan:

- Secure access
  - Authorization
  - Signed URL
  - Access control
  - File type validation
  - File size validation
  - Virus/malware scanning jika diperlukan
- 

## 63. SECURITY APPLICATION

Minimal mencakup:

- Authentication
  - Authorization
  - RBAC
  - Password security
  - Session security
  - API authentication
  - Input validation
  - File validation
  - Rate limiting
  - Secure endpoints
  - Audit trail
  - OWASP common protections
  - Secret management
  - No secret dalam log
- 

## 64. CERTIFICATE SECURITY

Public certificate verification tidak boleh menggunakan ID database sederhana yang mudah ditebak.

Gunakan:

**Secure Verification Token**

Contoh konsep:

```
graph TD
    Certificate --> UniqueNumber[Unique Number]
    UniqueNumber --> Plus[+]
    Plus --> SecureToken[Secure Token]
    SecureToken --> PublicVerificationURL[Public Verification URL]
```

---

## 65. QUEUE ARCHITECTURE

Queue digunakan untuk pekerjaan berat:

- Import
- Export
- Odoo synchronization
- Certificate generation
- Email
- WhatsApp
- Report generation
- Video processing
- Background aggregation

Tujuannya agar halaman web tetap responsif.

---

## 66. CACHE

Redis dapat digunakan untuk:

- Cache
- Queue
- Rate Limiting
- Session jika diperlukan

Namun:

**PostgreSQL tetap menjadi source of truth.**

---

## 67. OBSERVABILITY

Sistem harus dapat dipantau.

Minimal:

- Application logs
- Queue monitoring
- Error tracking
- Database monitoring
- CPU
- Memory
- Disk
- Response time
- Queue latency
- Failed jobs

Dapat dikembangkan menggunakan:

- Laravel Horizon
- Prometheus
- Grafana
- APM

---

## 68. LOAD TESTING

Sebelum production, lakukan load testing.

Tool yang dapat digunakan:

**k6**

Test minimal:

1. Login
  2. Dashboard
  3. Course listing
  4. Course detail
  5. Lesson access
  6. Progress update
  7. Assessment
  8. Certificate
  9. API Odoo
  10. Import processing
-

## 69. DATA CONSISTENCY

Proses kritis harus atomic.

Contoh:

```
Principal Change
  ↓
Employment History
  ↓
Principal History
  ↓
Current User
  ↓
Assignment
  ↓
Notification
```

Database transaction digunakan untuk perubahan data utama.

---

## 70. ADMIN MENU

Struktur menu yang disarankan:

```
Dashboard

User Management
├─ Users
├─ Import User Create
├─ Import User Update
└─ Export Users

Principal Management
├─ Principals
├─ Principal History
└─ Import / Export

Organization
├─ Positions
├─ Departments
└─ Import / Export
```

Course Management  
└─ Courses  
└─ Modules  
└─ Lessons  
└─ Assignment Rules  
└─ Import Create  
└─ Import Update  
└─ Export

Enrollment

Assessment  
└─ Question Bank  
└─ Questions  
└─ Assessments  
└─ Results

Certificate  
└─ Templates  
└─ Designer  
└─ Certificates

Notification  
└─ Templates  
└─ Settings  
└─ Logs

Reports

Integration  
└─ Odoo  
└─ Sync Logs  
└─ API

System  
└─ Roles  
└─ Permissions  
└─ Import Logs  
└─ Audit Logs

---

## 71. LEARNER MENU

Home

My Courses

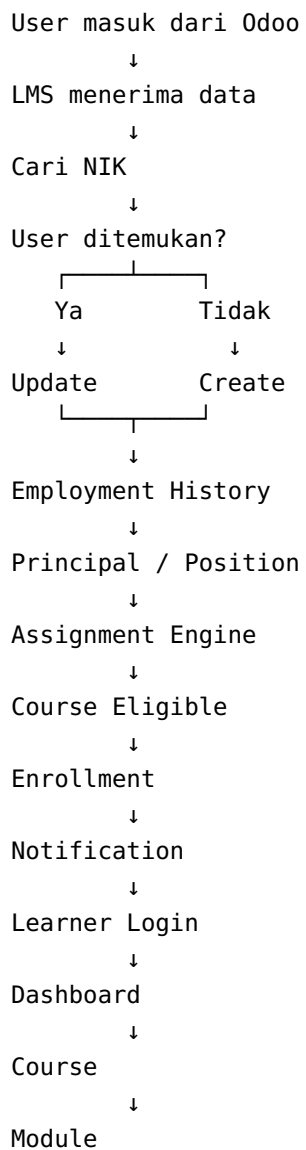
My Learning History

My Certificates

Profile

## 72. END-TO-END USER FLOW

Contoh proses dari awal:



↓  
Lesson  
↓  
Progress  
↓  
Post-Test  
↓  
Passed  
↓  
Certificate Generated  
↓  
Notification  
↓  
Certificate Available

---

## 73. CONTOH SKENARIO PRINCIPAL CHANGE

User:

NIK: 123456  
NIP: AMK001  
Principal: Xiaomi  
Position: Team Leader

Course:

Xiaomi Team Leader Training

User menyelesaikan course.

Certificate:

Principal: Xiaomi  
NIP: AMK001

Kemudian pindah:

NIP: AKP099  
Principal: Samsung  
Position: Team Leader

Sistem:

- User tetap sama.
- Employment lama menjadi historical.
- Principal Xiaomi menjadi inactive.
- Principal Samsung menjadi active.
- Course Samsung dievaluasi.
- Course yang eligible dibuat enrollment.
- Certificate Xiaomi tidak berubah.
- Learning History Xiaomi tetap ada.

---

## 74. KONSEP DATA HISTORY

Data berikut tidak boleh otomatis dihapus:

- Principal History
- Employment History
- Enrollment
- Learning Progress
- Learning History
- Assessment Results
- Certificates
- Audit Logs

Karena history merupakan bagian penting dari sistem.

---

## 75. DEVELOPMENT APPROACH

Implementasi jangan langsung membuat semua modul sekaligus.

Disarankan bertahap.

### Phase 0 — Requirement & Architecture

Finalisasi:

- Business rules

- Database
- API
- UI
- Security
- Performance
- Integration

## **Phase 1 — Identity & Organization**

Implement:

- Authentication
- Users
- Principals
- Positions
- Departments
- Employment History
- Principal History
- RBAC
- Import Create
- Import Update
- Export
- Audit

## **Phase 2 — Course**

Implement:

- Course
- Module
- Lesson
- Materials
- Principal-Course
- Assignment Rules

## **Phase 3 — Learning**

Implement:

- Enrollment
- Progress
- Sequential learning
- Resume
- Learning History

## **Phase 4 — Assessment**

Implement:

- Question Bank
- Questions
- Pre-Test
- Post-Test
- Scoring
- Passing Grade

## **Phase 5 — Certificate**

Implement:

- Template
- Designer
- Dynamic Fields
- PDF
- Image
- QR
- Public Verification
- Historical Snapshot

## **Phase 6 — Notification**

Implement:

- Email
- WhatsApp
- Templates
- Settings
- Queue
- Logs

## **Phase 7 — Odoo Integration**

Implement:

- API Adapter
- Sync
- Mapping
- Sync Logs
- Retry
- Failure Handling

## Phase 8 — Reporting

Implement:

- User Progress
- Course Progress
- Principal Report
- Learning History
- Assessment
- Certificate

## Phase 9 — Optimization

Implement:

- Cache
- Index Optimization
- Query Optimization
- Queue Optimization
- Load Testing
- Monitoring

## Phase 10 — Production

Implement:

- Deployment
- Backup
- Monitoring
- Security Review
- Disaster Recovery
- Documentation

---

# 76. REQUIREMENTS TRACEABILITY

Setiap requirement harus memiliki ID.

Contoh:

```
REQ-USER-001
REQ-USER-002
REQ-COURSE-001
REQ-ASSESS-001
```

REQ-CERT-001  
REQ-OD00-001

Setiap requirement dipetakan:

```
Requirement
  ↓
Module
  ↓
Database Entity
  ↓
API
  ↓
UI
  ↓
Implementation
  ↓
Test
  ↓
Security
  ↓
Performance
```

Tidak boleh ada requirement yang hilang tanpa status.

Jika belum dikerjakan:

**TODO**

Jika tidak disetujui:

**REJECTED**

Jika sudah selesai:

**COMPLETED**

---

## 77. TESTING

Minimal:

## Unit Test

Business logic.

## Feature Test

Fungsi aplikasi.

## Integration Test

Odoo API, Email, WhatsApp, Storage.

## Security Test

Authorization dan access control.

## Import Test

Create/Update validation.

## Performance Test

Load dan concurrency.

## Regression Test

Memastikan perubahan baru tidak merusak fitur lama.

---

# 78. BACKUP

Backup minimal:

- PostgreSQL
- Object Storage
- Configuration
- Encryption keys/secrets yang diperlukan

Backup harus memiliki:

- Schedule
- Retention
- Verification

- Restore test

Backup tanpa restore test tidak cukup.

---

## 79. DEPLOYMENT ENVIRONMENT

Minimal pisahkan:

```
Development
  ↓
Testing
  ↓
Staging
  ↓
Production
```

Database juga dipisahkan.

Jangan menggunakan production database untuk development.

---

## 80. DEVELOPMENT ENVIRONMENT

Untuk tahap awal:

```
Windows
|
├── Laragon
|   ├── PHP
|   ├── Nginx
|   └── Composer
|
├── Node.js
├── PostgreSQL
├── Redis
├── Docker
├── FFmpeg
├── Git
└── Antigravity
```

## 81. GIT

Repository harus dibuat sebelum development besar dimulai.

Konsep branch:

```
main
|
└─ development
    |
    ├── feature/user-management
    ├── feature/course
    ├── feature/assessment
    └─ feature/certificate
```

`main` harus selalu berada dalam kondisi stabil.

---

## 82. HAL YANG TIDAK BOLEH DILAKUKAN

### 1. Jangan membuat duplicate user karena NIP berbeda.

Identity = NIK.

- 2. Jangan menghapus history ketika user pindah Principal.**
- 3. Jangan menjadikan Odoo sebagai dependency untuk setiap request LMS.**
- 4. Jangan membuat Import Create dan Update menjadi satu fungsi.**
- 5. Jangan memproses import besar dalam satu HTTP request.**
- 6. Jangan menyimpan video besar langsung di database.**
- 7. Jangan menjalankan video streaming melalui Laravel application server.**
- 8. Jangan melakukan query report berat setiap kali dashboard dibuka.**
- 9. Jangan menyimpan progress video setiap detik ke database.**
- 10. Jangan membuat microservices hanya karena terlihat lebih advanced.**
- 11. Jangan mengorbankan data history demi simplifikasi.**
- 12. Jangan membuat business rule hanya di frontend.**

Business rule penting harus tetap divalidasi backend dan database.

---

## **83. KEPUTUSAN YANG HARUS DIDISKUSIKAN SEBELUM CODING**

Ini bagian paling penting untuk meeting.

Sebelum Antigravity mulai coding, kita harus menyepakati:

### **A. Identity**

1. Apakah NIK benar-benar mandatory?
2. Apakah NIK selalu UNIQUE?

3. Bagaimana menangani NIK yang salah?
4. Siapa yang boleh melakukan Identity Correction?

## **B. Employment**

1. Apakah hanya boleh satu Employment ACTIVE?
2. Apakah seseorang boleh memiliki lebih dari satu employment aktif dalam kondisi tertentu?
3. Bagaimana menentukan current employment?

## **C. Principal**

1. Apakah satu user hanya boleh memiliki satu Principal aktif?
2. Apa yang terjadi ketika pindah Principal?
3. Apakah course lama menjadi `PRINCIPAL INACTIVE` ?

## **D. Assignment**

1. Apakah semua course Principal harus menggunakan Assignment Rule?
2. Apakah ALL POSITION diperlukan?
3. Apakah Department ikut menentukan assignment?
4. Apakah assignment memiliki expiry?

## **E. Learning**

1. Apakah semua lesson wajib sequential?
2. Apakah video wajib 100% untuk complete?
3. Apakah playback speed harus dikunci?
4. Apakah user boleh mengulang course?

## **F. Assessment**

1. Apakah Pre-Test wajib?
2. Apakah Post-Test wajib?
3. Berapa kali boleh retake?
4. Apakah score tertinggi atau score terakhir yang digunakan?

## **G. Certificate**

1. Kapan certificate diterbitkan?
2. Apakah certificate memiliki expiry?
3. Siapa yang dapat membuat template?
4. Apakah certificate perlu approval?

## **H. Notification**

1. Email provider apa?
2. WhatsApp provider apa?
3. Event apa yang wajib dikirim?
4. Apakah user dapat disable notification tertentu?

## I. Odoo

1. API Odoo apa yang tersedia?
2. Data apa yang menjadi master Odoo?
3. Data apa yang menjadi master LMS?
4. Berapa interval synchronization?
5. Apakah sync realtime atau scheduled?

## J. Infrastruktur

1. Berapa estimasi jumlah user?
2. Berapa estimasi concurrent user?
3. Berapa volume video?
4. Berapa pertumbuhan data per tahun?
5. Apakah LMS akan digunakan hanya internal atau dapat diakses eksternal?

---

# 84. ESTIMASI SCALE YANG PERLU DITENTUKAN

Sebelum menentukan kapasitas server, kita perlu mengetahui minimal:

```
Total User
Active User
Daily Active User
Peak Concurrent User
Course Count
Lesson Count
Video Storage
Average Video Size
Assessment Volume
Certificate Volume
Notification Volume
Import Volume
```

Contoh:

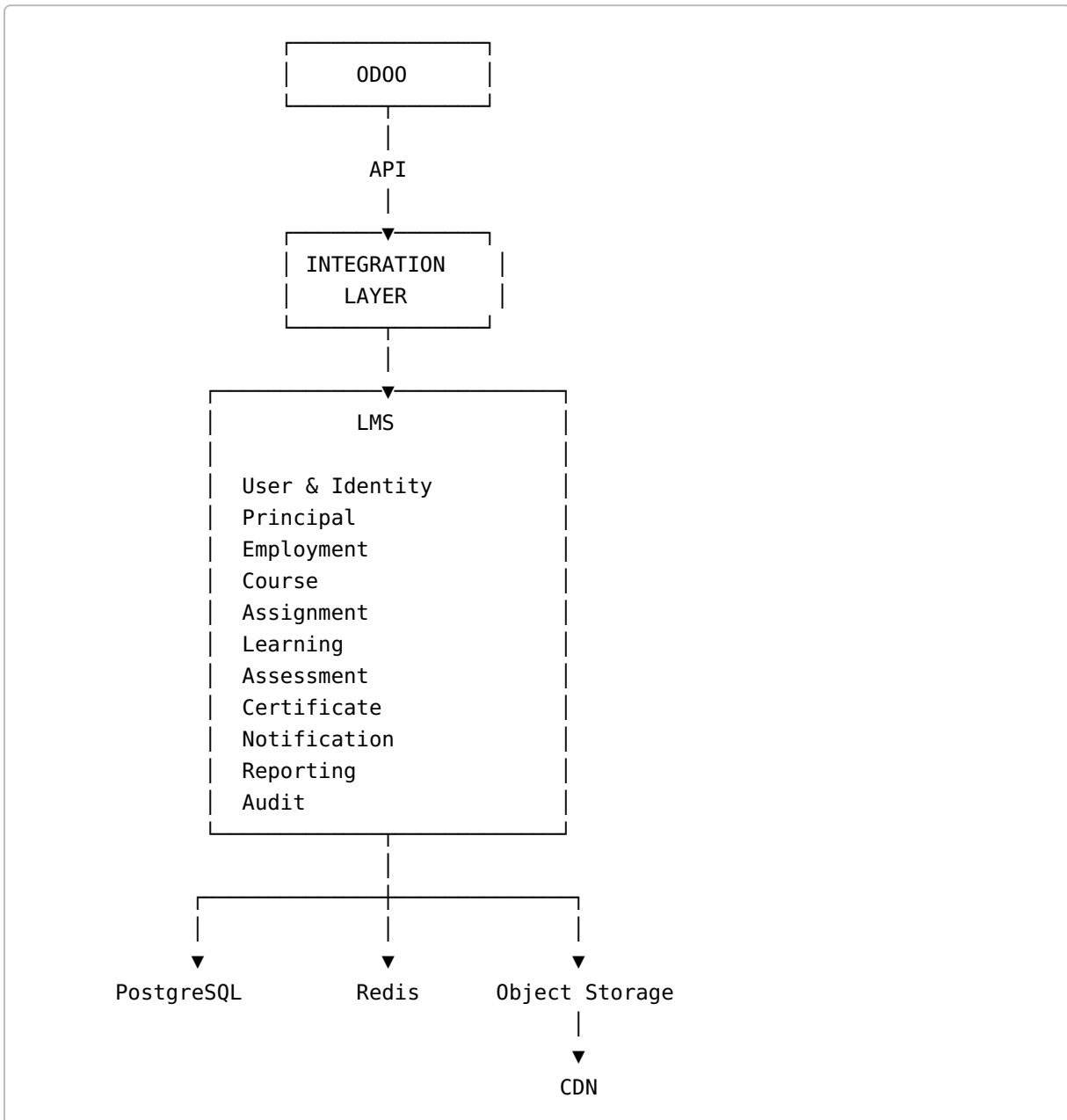
```
Users           : 20,000
DAU             : 8,000
Peak Concurrent : 1,000
Courses        : 500
Lessons         : 5,000
Video Storage   : 2 TB
```

Angka aktual harus ditentukan berdasarkan kebutuhan perusahaan.

---

## 85. KONSEP AKHIR SISTEM

Secara sederhana:



## 86. KONSEP BISNIS DALAM SATU KALIMAT

LMS adalah sistem standalone yang mengelola satu identitas user berdasarkan NIK, memiliki riwayat employment dan Principal, memberikan course berdasarkan eligibility/assignment rule, mencatat seluruh aktivitas pembelajaran, assessment, certificate, notification, dan history, serta menerima data eksternal seperti Odoo melalui API tanpa menjadikan Odoo sebagai dependency utama.

---

## 87. REKOMENDASI KEPUTUSAN TEKNIS AWAL

Untuk diskusi, baseline yang direkomendasikan:

| Komponen         | Teknologi                    |
|------------------|------------------------------|
| Backend          | Laravel 13                   |
| PHP              | PHP 8.4+                     |
| Frontend         | Vue 3                        |
| Language         | TypeScript                   |
| Build            | Vite                         |
| CSS              | Tailwind CSS                 |
| Database         | PostgreSQL                   |
| Cache            | Redis                        |
| Queue            | Laravel Queue                |
| Queue Monitoring | Laravel Horizon              |
| Web Server       | Nginx                        |
| Video Processing | FFmpeg                       |
| Video Streaming  | HLS                          |
| File Storage     | S3-compatible Object Storage |
| CDN/WAF          | Cloudflare                   |
| Container        | Docker                       |
| Monitoring       | Prometheus/Grafana/APM       |
| Load Test        | k6                           |
| Architecture     | Modular Monolith             |

| Komponen    | Teknologi |
|-------------|-----------|
| Integration | REST API  |

## 88. KEPUTUSAN PALING PENTING

Sebelum masuk Antigravity, tim sebaiknya mengunci 10 hal berikut terlebih dahulu:

### 1. Identity

**NIK = UNIQUE PERSON ID**

### 2. Employment

**NIP = Employment Identity, bukan User Identity**

### 3. Principal

**Satu user = satu Principal ACTIVE**

### 4. History

**History tidak dihapus ketika terjadi perubahan**

### 5. Assignment

**Course diberikan berdasarkan Assignment Rule, bukan otomatis semua course Principal**

### 6. LMS

**LMS standalone dan memiliki database sendiri**

### 7. Odoo

**Odoo hanya salah satu source melalui API**

### 8. Import

**Import Create dan Import Update dipisahkan**

### 9. Performance

**Queue + Redis + PostgreSQL + Object Storage + CDN**

## 10. Development

Mulai dari requirement → architecture → database → Phase 1, bukan langsung generate seluruh aplikasi

---

# 89. URUTAN DISKUSI DENGAN REKAN

Agar meeting tidak melebar, gunakan urutan:

### Meeting 1 — Business Rules

Bahas:

- User
- NIK
- NIP
- Principal
- Employment
- Course assignment
- Learning
- Assessment
- Certificate

### Meeting 2 — Data & Integration

Bahas:

- Database
- Odoo
- Import
- Export
- History
- Source ownership

### Meeting 3 — Technical Architecture

Bahas:

- Laravel
- Vue
- PostgreSQL
- Redis
- Storage
- Video
- Queue

- CDN
- Security

## Meeting 4 — Implementation

Bahas:

- Phase
- Prioritas
- Timeline
- Developer responsibility
- Testing
- Deployment

Setelah empat tahap tersebut disepakati, barulah requirement final dimasukkan ke AntigraVity.

---

## 90. OUTPUT YANG HARUS DIMINTA DARI ANTIGRAVITY

AntigraVity **jangan langsung diminta membuat seluruh aplikasi.**

Tahap pertama harus meminta AntigraVity menghasilkan:

1. Requirement Analysis
2. Architecture Proposal
3. Technology Decision
4. Database Design
5. Entity Relationship Diagram
6. API Architecture
7. API Endpoint Specification
8. Authentication Architecture
9. RBAC Architecture
10. Import/Export Architecture
11. Assignment Engine Design
12. Learning Progress Architecture
13. Certificate Architecture
14. Notification Architecture
15. Odoo Integration Architecture
16. Security Architecture
17. Performance Architecture
18. Queue Architecture
19. File/Video Storage Architecture
20. Reporting Architecture
21. Folder Structure
22. Requirements Traceability Matrix

- 23. Development Roadmap
- 24. Testing Strategy
- 25. Deployment Strategy

Setelah hasil tersebut direview dan disetujui manusia, barulah Antigravity mulai implementasi.

---

## 91. KESIMPULAN

Konsep LMS ini bukan sekadar aplikasi untuk menampilkan video dan memberikan sertifikat.

Sistem yang dibangun adalah **platform Learning Management multi-Principal** yang memiliki tiga lapisan utama:

IDENTITY & ORGANIZATION  
+  
LEARNING MANAGEMENT  
+  
INTEGRATION & OPERATION

Dengan konsep:

NIK  
↓  
ONE USER  
↓  
EMPLOYMENT HISTORY  
↓  
PRINCIPAL / POSITION  
↓  
ASSIGNMENT RULE  
↓  
COURSE  
↓  
MODULE  
↓  
LESSON  
↓  
PROGRESS  
↓  
ASSESSMENT  
↓  
CERTIFICATE

↓  
LEARNING HISTORY

Sedangkan sumber datanya:

Odoo  
Manual  
Import  
Future API  
↓  
Integration Layer  
↓  
LMS

Dan fondasi teknisnya:

Laravel  
+  
Vue  
+  
PostgreSQL  
+  
Redis  
+  
Queue  
+  
Object Storage  
+  
CDN  
+  
Docker

Target akhirnya adalah LMS yang:

- Standalone
- Multi-Principal
- Scalable
- Fast
- Responsive
- Secure
- Auditable
- API-first
- Import/Export capable
- Odoo-integrated
- History-preserving

- Siap dikembangkan dalam jangka panjang.

**Prinsip utama: jangan mulai dari coding. Mulai dari menyepakati business rules dan data model. Setelah itu baru arsitektur, kemudian implementasi.**